Lctr 9: Preemption, EXTI, and Finite State Machine

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Priority of interrupt and preemption. Principle and configuration of:
  - Preemption priority.
  - Sub-priority.
- EXTI (external interrupt) with GPIO pins.
- Finding callback functions.
- Using finite state machines to handle different cases.

Demo project:

- Using EXTI0 to avoid polling.
- Preemptive interrupts with EXTI0 and UART2.

Additional reading:

- Configuring EXTI in CubeMX.
- Tail-chaining of interrupts.

9.1 Introduction

9.2 Demo project

9.2.1 Essential GPIO pins of the project

This project uses two GPIO pins on G431n32:

- PA0, labeled as Fnc6, for input as EXTI0.
- PB8, labeled as LED4, for GPIO output.

9.2.2 Interrupts involved in the project

Here are the basic ideas for interrupt control:

- We use two different interrupts, EXTI0 and UART2, to illustrate the preemption priority (level).
  - We set up UART2 with medium preemption priority.
  - We set up EXTI0 with changeable preemption priority. When it is low, its interrupt handler can be preempted by the handler of UART2.
- We set up the SysTick timer with the highest preemption priority. This way, it will not be affected by the handlers of EXTI0 and UART2.

9.2.3 Blinking patterns

To see the preemption of EXTI0 by UART2, we use different blinking patterns of the LED in the main program and the **exaggerated** callback functions:

- In the main program, the LED will blink at a duty cycle of 50% with a period of 1 second (on for 500 ms and off for 500 ms).
- In the callback function of EXTI0, it will blink 5 times with the following time periods: on for 200 ms and off for 800 ms. This is a total of 5 seconds.
- In the callback function of UART2, it will blink 5 times with the following time periods: on for 400 ms and off for 100 ms. This is a total of 2.5 seconds.

9.3 Priority of interrupt and preemption

We can configure the priority of an IRQn by setting a priority number, as discussed below.

9.3.1 Priority levels and priority numbers

ARM processors use reversed priority relationship—a smaller priority number value means a higher priority (level) or higher urgence. For example, suppose we have two interrupts, the priority number of which are 2 and 5, respectively. Then, the one with priority number 2 has higher priority than the one with 5.

9.3.2 Preemption priority and sub-priority

Like many other modern processors, ARM processors have two different types of priorities: the **preemption priority**, also called group priority, and the **sub-priority**.

As such, the priority number (PN) of an interrupt is the combination of both the preemption priority number (PPN) and the sub-priority number (SPN).

9.3.2.1 Preemption priority

Preemption priority allows an interrupt with a higher preemption priority (smaller PPN value) to preempt the running ISR of an interrupt with lower preemption priority (greater PPN value). If two interrupts are on the same preemption priority (same PPN value), they cannot preempt each other.

Suppose we have Interrupts A and B. **Assume A's PPN value is smaller than that of B** (A can preempt B and B cannot preempt A). We can see the following process of the program's running:

1. The main program runs.
2. Interrupt B triggers, and the processor stops the main program to run the ISR of B.
3. Before B's ISR runs to the end, Interrupt A triggers. Then, the processor will halt the ISR of B to run the ISR of A. Now, B is preempted by A.
4. When the ISR of A runs to the end, the processor will resume the running of the ISR of B, which can run to the end if A does not trigger again.
5. The processor then resumes the running of the main program.

We can also see another process of the program's running:

1. The main program runs.
2. Interrupt A triggers, and the processor stops the main program to run the ISR of A.
3. Before A's ISR runs to the end, Interrupt B triggers. Since interrupt B has lower preemption priority than Interrupt A, the ISR of A will keep running.
4. When the ISR of A runs to the end, the processor will start the running of the ISR of B, which can run to the end if A does not trigger again.
5. The processor then resumes the running of the main program.

9.3.2.2 Sub-priority

When we cannot service an interrupt, it will be pending, waiting to be serviced.

- If we have several pending interrupts, the one with smallest PPN value will be serviced once the processor is available.
- If pending interrupts have the same PPN value, the one with highest sub-priority (**smallest SPN** value) will be serviced first.

9.3.2.3 Tied priorities

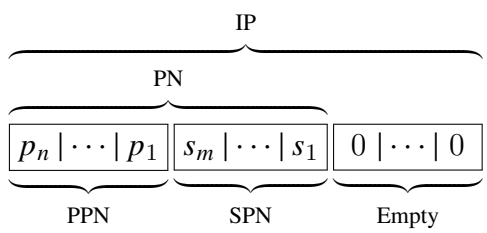
If two interrupts with exactly the same PPN and SPN values are pending, the one with lower position (index) in the NVIC table, which means smaller IRQn value, will be serviced first.

9.3.3 The interrupt priority register

The IPs are defined in an array of 240 as shown below in `core_cm4.h` since it is the same for all Cortex-M4 processors:

```
typedef struct {
    // ...
    __IO unit8_t IP[240];  // Interrupt Priority Registers
    // ...
} NVIC_Type;
```

Most manufactures do not implement all the 8 bits. In the case shown below, we have n bits for PPN, m bits for SPN, and $8 - n - m$ bits empty. Note that **these empty bits default to 0 when the register is read**.



We can see that given PPN, SPN, and n and m, we can easily calculate PN as

$$\text{PN} = (\text{PPN} << m) + \text{SPN}.$$

Note that the value of $n + m$ is determined by the implementation of the hardware, and the value of n can be configured, as discussed below.

9.3.4 Setting the priority grouping

The setup of the value of numbers of bits for PPN, n, which can be 0, is commonly referred to as splitting the group priority (preemption priority) and sub-priority. This is controlled by bits [10:8] of AIRCR (Application Interrupt and Reset Control Register). The control is given below:

G = bits [10:8]	Group priority field	Sub-priority field
0	n = 7 bits, [7:1]	[0]
1	n = 6 bits, [7:2]	[1:0]
2	n = 5 bits, [7:3]	[2:0]
3	n = 4 bits, [7:4]	[3:0]
4	n = 3 bits, [7:5]	[4:0]
5	n = 2 bits, [7:6]	[5:0]
6	n = 1 bit, [7]	[6:0]
7	n = 0 bit	[7:0]

- It is common not to implement all 8 bits in IP.
- If only bits `[7:4]` of IP are implemented, the values of G = 0 to 3 will have the same effect.
- We often use macros to define the value of n, as shown in the following definition provided in `stm32g4xx_hal_cortex.h`:

```
#define NVIC_PRIORITYGROUP_0 7U // 0 bit for PPN, 4 for SPN
#define NVIC_PRIORITYGROUP_1 6U // 1 bit for PPN, 3 for SPN
```

Usually, the value of n is already set up in the code generated by CubeMX. If we want to change the setup in user code, we will use the following function:

```
NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_n);
```

where `n` is a specific value as shown in the above code snippet.

9.3.5 Setting the priority number for an IRQn

As we discussed before, the priority number (PN) is the combination of both the preemption priority number (PPN) and the sub-priority number (SPN), the numbers of the preemption priority field and the sub-priority fields.

With the priority number, the priority of an IRQn can be changed easily in the program. This can be done by calling the following NVIC function:

```
NVIC_SetPriority(IRQn, priority_number);
```

Below, we use two examples to illustrate calculations using the above numbers.

Example 1. Consider the setup of the priority number of UART2. Assume we have the following:

- There are 4 bits implemented for the IP register.
- Two bit are used for preemption priority group.
- The PPN and SPN should be PPN = 2 and SPN = 1, respectively.

Determine the value of PN to set up the IP using the NVIC function. Write the code to set it up. What will be the value of the corresponding IP (in hexadecimal) if we read it out.

Solution.

The PN is calculated as

```
PN = (PPN << 2) + SPN = 8 + 1 = 9.
```

To set it up, call:

```
NVIC_SetPriority(USART2_IRQn, 9);
```

The read out value of the IP will be `9 << (8-4)= 0x90`.

> :-:

Example 2. Consider the problem of Example 1 again.

1. If we want to set up the priority number of EXTI0 so that it can preempt the ISR of UART2. What will be the range (inclusive) of this priority number?
2. Consider Part 1 again. What if we do not want EXTI0 to preempt UART2?

Solution.

According to Example 1, the PPN of UART2 is 2.

1. For EXTI0 to be able to preempt UART2, its PPN should be 0 or 1. Its SPN can be 0 to 3. To find the range, we need to use the smallest values of both, `0b00_00`, to get the lower bound of the range and the greatest values of both, `0b01_11`, to get the upper bound of the range. So the PN of EXTI0 is from 0 to 7.
2. For EXTI0 not to be able to preempt UART2, its PPN should be no less than 2. Its SPN can be 0 to 3. To find the range, we need to use the smallest values of both, `0b10_00`, to get the lower bound of the range and the greatest values of both, `0b11_11`, to get the upper bound of the range. So the PN of EXTI0 is from 8 to 15.

> :-:

9.3.6 Priority setups for the demo project

- 4 bits total for PPN and SPN.
- 2 bits for the priority group (PPN).
- Highest priority for SysTick timer with PPN = 0 and SPN = 0. This way, it will not be affected by the handlers of EXTI0 and UART2.
- Fixed priority for UART2 with PPN = 1 and SPN = 0. (The priority number is 4.)
- Variable priority for EXTI0 with initial values: PPN = 2 and SPN = 0; now, the priority number is 8. We can change it in the program according to user input from UART.
  - We can change the priority of EXTI0 to PPN = 1 and SPN = 0, which has a priority number of 4. In this case, it has the same priority as UART2, and hence we will not be able to preempt it using UART2.

- We can also change the priority number back to the original value 8. In this case, it has a lower priority than UART2, and hence we can preempt it using UART2.

9.4 Extended Interrupt/Event Controller

With EXTI (extended interrupt/event controller), we can use GPIO inputs to trigger interrupts. Hence, we don't need to poll the level of GPIO inputs any more; the change of the input levels can be detected by the EXTI controller.

Figure 1 shows a diagram of the EXTI (extended interrupt/event controller) of STM32L476. Note that *Configurable events* can be connected to GPIO pins via multiplexers that will be shown later.

The extended interrupt/event block diagram is shown on *Figure 33*.

Figure 33. Configurable interrupt/event block diagram

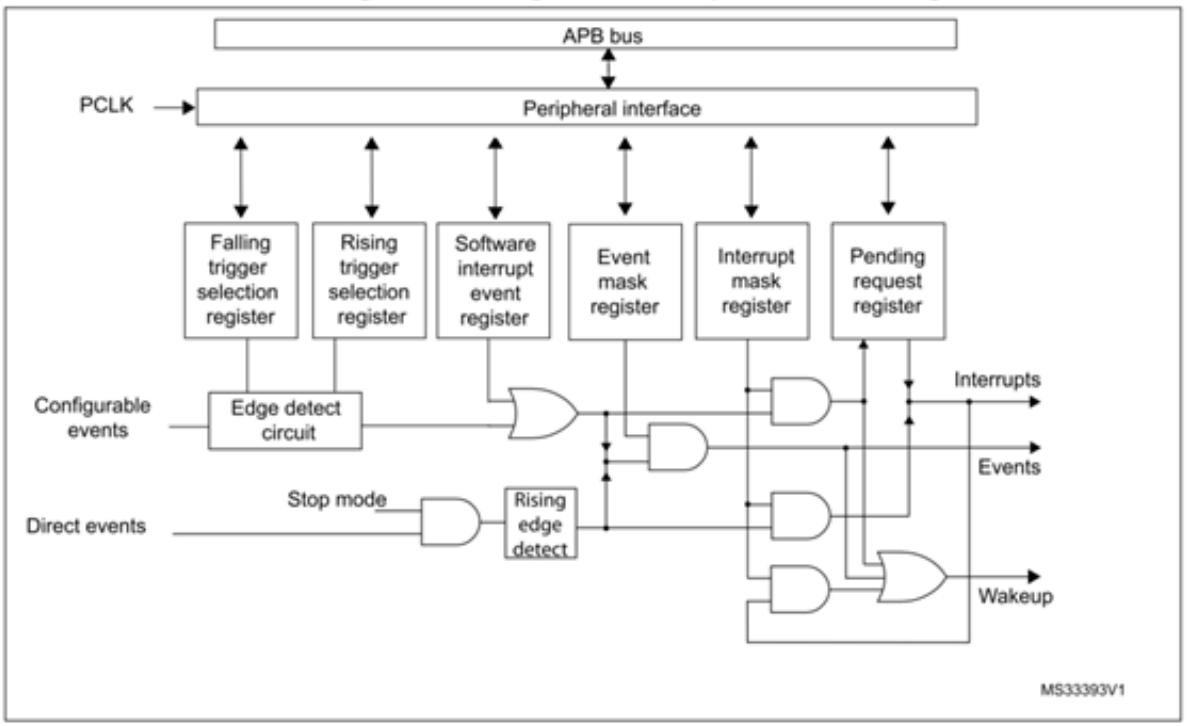


Figure 1: Diagram of the extended interrupt/event controller of STM32L476.

In the diagram, the following registers are related to GPIO inputs:

- Falling (edge) trigger selection register: use falling edge detection on an external input from GPIO.
- Rising (edge) trigger selection register: use rising edge detection on an external input from GPIO.

9.4.1 Multiplexing GPIO pins to EXTI

9.4.1.1 Pin multiplexing

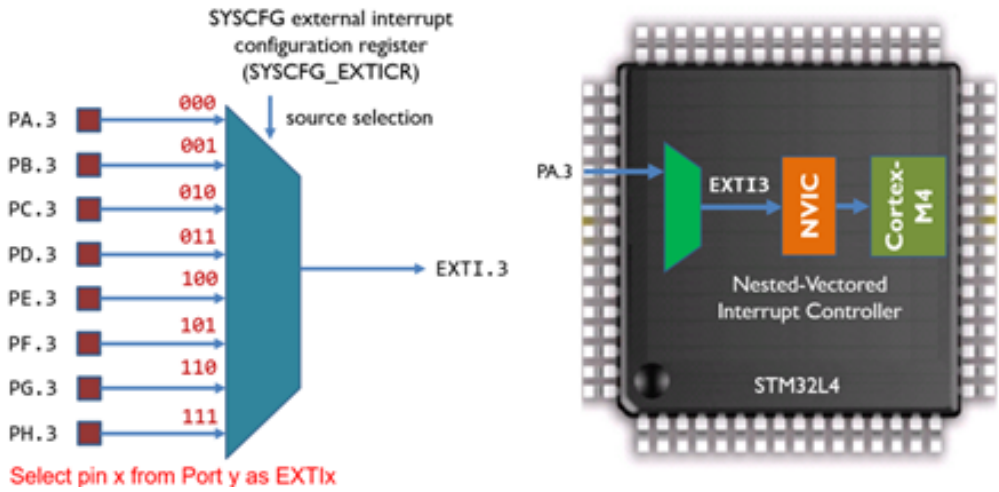
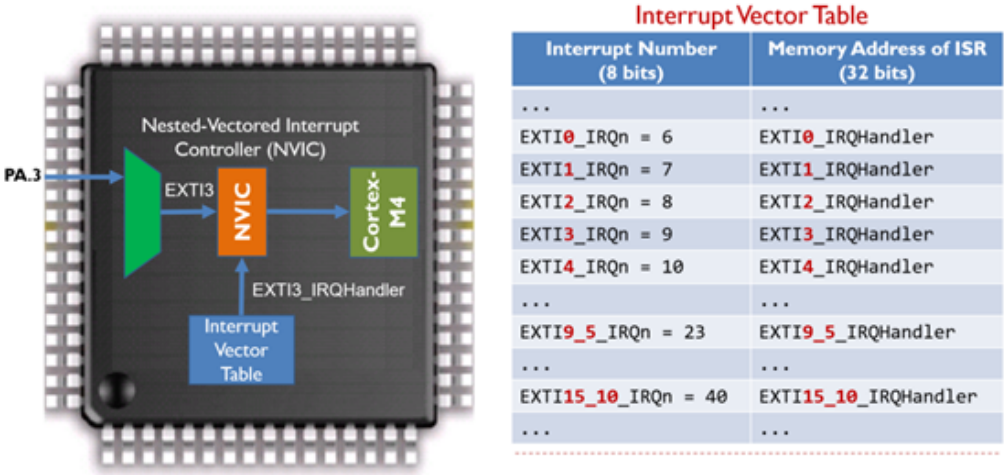


Figure 2: EXTI output from multiplexing the same pin from all GPIO ports.

9.4.1.2 Pin combination

Pin combination uses an OR gate to combine multiple outputs of the pin multiplexers into one output. For example we can combine GPIO Pins 5 to 9 into EXTI9_5 and GPIO Pins 10 to 15 into EXTI15_10.



Interrupt Number (8 bits)	Memory Address of ISR (32 bits)
...	...
EXTI0_IRQn = 6	EXTI0_IRQHandler
EXTI1_IRQn = 7	EXTI1_IRQHandler
EXTI2_IRQn = 8	EXTI2_IRQHandler
EXTI3_IRQn = 9	EXTI3_IRQHandler
EXTI4_IRQn = 10	EXTI4_IRQHandler
...	...
EXTI9_5_IRQn = 23	EXTI9_5_IRQHandler
...	...
EXTI15_10_IRQn = 40	EXTI15_10_IRQHandler
...	...

Figure 3: EXTI output from multiplexing the same pin from all GPIO ports.

9.4.2 Setting up an EXTI via using SYSCFG registers

Much like we can set up GPIO pins via direct GPIO register access, we can set up EXTIs via using the SYSCFG (system configuration) registers and EXTI registers. The details are omitted here due to space limit; we just show the following (semi-pseudo) code to set up EXTI0 using PA0:

```
// 1. Setting up PA0 as GPIO input as for common GPIO pins.

// 2. Setting up the multiplex for PA0
// 2.1 Enabling the clock for SYSCFG
__HAL_RCC_SYSCFG_CLK_ENABLE();
    // wrapper for "RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN"

// 2.2 Choosing PA0 using bitwise operation
SYSCFG->EXTICR[0] &= ~MASK_FOR_EXTI0;    // Use a mask
SYSCFG->EXTICR[0] |= VALUE_FOR_EXTI0_PA; // Assign value

// 3. Setting up the mode of EXTI0
// 3.1 RTSR (rising edge trigger selection register):
    // disabled: 0, enabled: 1
EXTI->RTSR1 |= EXTI_RTSR1_RT0;

// 3.2 IMR (interrupt mask reg):
    // masked (disabled): 0, not masked (enabled): 1
EXTI->IMR1 |= EXTI_IMR1_IM0;

NVIC_EnableIRQ(EXTI0_IRQn);
```

9.5 Finding callback functions in the provided interrupt handler

- An ISR is generated for **each specific MPB** by CubeMX. The file containing the ISR is MPB specific, and hence if we add user functions there, we need to change multiple ISRs for all the boards supported by the project.
- A callback function is a generic term comes from event-driven programming that describes a *user* function that is called to provide specific user logics when a certain event happens. This function is generated as a *weak* function and a true user function can be defined anywhere.

See the Collection for how to find the callback function for EXTI0_IRQn using the demo project.

9.6 Finite state machine (FSM)

In programming, a **state** is defined as a status of a function based on which the function behaves in a specific way. The state can transition depending on some conditions. Usually, we consider the case that the state has only a finite number of status, and the function is called a **finite state machine** (FSM).

9.6.1 A basic FSM for implementing UART2's callback function

To illustrate the usage of an FSM, let's consider a fairly complex callback function, which has the following functionalities.

- When the program first starts, we need to press any key in PuTTY to move forward. This input will not trigger the blinking of the LEDs in UART2's handler. This is the initial state of the FSM, called `STATE_INI`. In this state, we need to read out the priority number setup of the EXTI0 from the corresponding IP register to determine the next state:
  - If the priority number is no less than 8 (corresponding to `(2 << 2) + 0`), then we move to the `STATE_PREEMPT_YES` state, as EXTI0 has lower preemption priority than UART2.
  - Otherwise, we move to the `STATE_PREEMPT_NO` state and set up the priority number of EXTI0 to 4.
- When the program works at the `STATE_PREEMPT_YES` state, it scans each keyboard input. If the reading is 'n', then it changes the state to `STATE_PREEMPT_NO` and changes the priority number of EXTI0 to 4. In any case, it runs a sequence of LED blinking, as described in the Blinking patterns section.
- When the program works at the `STATE_PREEMPT_NO` state, it scans each keyboard input. If the reading is `y`, then it changes the state to `STATE_PREEMPT_YES` and changes the priority number of EXTI0 to 8. It then runs the same sequence of LED as above.

9.6.2 FSM-based callback function of UART2

The code for FSM-based callback function for UART2 is given in `di2f_exit_n_uart_callback.c` together with other supporting code:

```c
// Callback for UARTs
// - Only handles UART2
typedef enum {
    STATE_INI = 0,       // Initial state
    STATE_PREEMPT_YES,   // State that EXTI0 can be preempted
    STATE_PREEMPT_NO,    // State that EXTI0 cannot be preempted
} state_t;

static state_t state = STATE_INI;

#define CRG_RETURN 13   // ASCII code for carriage return

extern uint8_t uart2_rxBuffer[];
#define PREEMPT_YES 4   // input of 4 will enable
#define PREEMPT_NO  8
char pmpt_PREEMPT_YES[] =
        "\nPreemption priority enabled. Input 'n' to disable.\n";
char pmpt_PREEMPT_NO[]  =
        "\nPreemption priority disabled. Input 'y' to enable.\n";
char cmd_core[] = "  ";
uint8_t priority = 0;
uint8_t new_input;

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    new_input = uart2_rxBuffer[0];

    switch (state) {
    case STATE_INI:
        priority = NVIC->IP[EXTI0_IRQn] >> 4;
        printf("EXTI0's priority is %d \n", priority);
        if (priority >= 8) {
            state = STATE_PREEMPT_YES;
            printf("%s\n", pmpt_PREEMPT_YES);
        } else {
```

```
            state = STATE_PREEMPT_NO;
            printf("%s\n", pmpt_PREEMPT_NO);
            NVIC_SetPriority(EXTI0_IRQn, 4);
        }
        break;

    case STATE_PREEMPT_YES:
        if (new_input == 'n') {
            priority = 4;
            NVIC_SetPriority(EXTI0_IRQn, priority);
            printf("Priority of EXTI0 is set to (%d, %d).\n",
                    priority>>2, priority&3U);
            state = STATE_PREEMPT_NO;
            printf("%s\n", pmpt_PREEMPT_NO);
        }
        task_4_UART2();  // Blink LED 5 times: on for 400 ms; off 100 ms
        break;

    case STATE_PREEMPT_NO:
        if (new_input == 'y') {
            priority = 8;
            NVIC_SetPriority(EXTI0_IRQn, priority);
            printf("Priority of EXTI0 is set to (%d, %d).\n",
                    priority>>2, priority&3U);
            printf("%s\n", pmpt_PREEMPT_NO);
            state = STATE_PREEMPT_YES;
        }
        task_4_UART2();
        break;
    }

    HAL_UART_Receive_IT(&huart2, uart2_rxBuffer, 1);
}
```